CUADERNO PARA PREPARAR TU EXAMEN

Práctica integradora

Python, explicado desde cero y línea por línea

Una guía para entender lo que hace cada instrucción, seguir los datos y poder explicar la solución con tus propias palabras.



Facundo Canellada · Programación I · 03 de noviembre de 2026

Basada en los 5 archivos de tu carpeta 13 - Practica Integradora: 243 líneas, incluyendo comentarios y líneas en blanco.

Tu recorrido de estudio

Podés leer de principio a fin o usar los capítulos como una consulta. El índice y los marcadores del PDF permiten saltar a cada parte.

1. Empezamos sin apurarnos 3

2. Las piezas básicas de Python 4

3. Qué significa cada palabra del código 6

4. Qué significa cada símbolo 12

5. El programa, línea por línea 15

Articulo.py . . . 16

RemitoVentaDetalle.py . . . 17

RemitoVenta.py . . . 18

RepartoDiario.py . . . 20

CrearRepartoDiario.py . . . 22

6. Cómo trabaja cada función 36

7. Un reparto completo, sin saltar pasos 38

8. Bucles y validaciones con ejemplos 41

9. Errores que conviene poder reconocer 43

10. Practicá como en el examen 44

11. Los cinco archivos completos 47

12. Cómo ejecutar y qué material se usó 54

PASO 01

1. Empezamos sin apurarnos

Vamos de a una idea por vez. El objetivo es que puedas mirar una línea, decir qué datos usa y anticipar qué ocurrirá después. No hace falta memorizar todo el programa de una sola vez.

Una imagen para pensar el problema

Imaginá un local que vende repuestos. El catálogo es una colección de fichas con productos y precios. Un cliente compra algunos productos: cada renglón de esa compra es un detalle. La hoja con todos esos renglones es el remito. La carpeta que reúne las hojas del día es el reparto.

En la vida cotidiana	En el programa
Ficha de un producto	Un objeto `Articulo`.
Un renglón: 2 filtros a $10000 cada uno	Un objeto `RemitoVentaDetalle` con subtotal $20000.
La hoja de venta de un cliente	Un objeto `RemitoVenta` con una lista de detalles.
La carpeta de las ventas del día	Un objeto `RepartoDiario` con una lista de remitos.

Cómo estudiar una línea

1. Leé la instrucción en voz alta: por ejemplo, «agrego este detalle al remito».
2. Buscá qué datos necesita: cantidad, precio, objeto o lista.
3. Preguntate qué cambia: una variable, un atributo, una lista o solamente la pantalla.
4. Seguí el camino: ¿continúa debajo, entra en un bloque, repite o vuelve a quien llamó?
5. Probá con números pequeños y anotá los resultados en papel.

> **Todavía no lo entiendo: ¿por dónde sigo?**
>
> Volvé al ejemplo más pequeño: un artículo de $10000 y una cantidad de 2. Cuando puedas explicar por qué el subtotal da $20000, agregá un segundo detalle. Después, agregá un segundo remito.

Qué vas a poder explicar

Qué es cada clase; qué recibe y devuelve cada función; por qué hay listas y un diccionario; cómo se validan los datos; qué hace cada uno de los tres bucles de carga; y cómo un importe viaja desde un detalle hasta el total del reparto.

PASO 02

2. Las piezas básicas de Python

Un nombre permite volver a encontrar un dato

```
cantidad = 2
precio = 10000
subtotal = cantidad * precio
```

A la derecha del signo = se obtiene un resultado. A la izquierda se indica con qué nombre lo vamos a usar. Después de estas instrucciones, `subtotal` permite consultar el número 20000. En Python un nombre se vincula a un objeto; cambiar el nombre que apunta a un objeto no crea automáticamente una copia.

Tipo de dato	Ejemplo	Qué significa
Cadena (`str`)	`"10"`	Texto. Se usa para códigos, nombres, CUIT y fecha.
Entero (`int`)	`2`	Número sin parte decimal. Se usa para cantidades y número de remito en esta carga.
Decimal binario (`float`)	`0.0`	Número con parte decimal. El total empieza así; mostrar dos decimales no vuelve exacta su representación interna.
Booleano (`bool`)	`True` / `False`	Resultado lógico: verdadero o falso.
Ausencia	`None`	Valor especial que aquí expresa que una búsqueda no encontró un artículo.

Clase, objeto, atributo y método

```
articulo = Articulo("10", "Filtro de Aceite", 10000)
precio = articulo.precio
```

La clase es el molde `Articulo`. El objeto es el artículo concreto creado por la llamada. El atributo `precio` es un dato que pertenece a ese objeto. Un método es una función definida dentro de una clase, como `remito.calcular_total()`.

Al escribir `self.precio = precio`, el nombre de la derecha es el parámetro recibido y el de la izquierda es el atributo del objeto actual. Pueden llamarse igual porque se distinguen por `self.`.

Parámetro y argumento

```
def leer_texto(mensaje):
    ...

fecha = leer_texto("Fecha del reparto: ")
```

En la definición, `mensaje` es un parámetro: un nombre preparado para recibir un dato. En la llamada, el texto "Fecha del reparto: " es el argumento concreto que se envía. Los puntos suspensivos del ejemplo indican contenido omitido: no hay que reemplazar con ellos el cuerpo real de la función.

Una lista guarda varios elementos

```
self.detallesRemito = []
self.detallesRemito.append(detalle)
```

La primera línea crea una lista vacía. La segunda incorpora un objeto a esa lista. El remito conserva los detalles completos, por eso después puede obtener tanto el subtotal como la denominación de cada artículo. La lista se crea en `__init__` para que cada remito tenga la suya.

Un diccionario relaciona una clave con un valor

```
clientesReparto = {"20284569875": "Juan Alonso"}
nombreCliente = clientesReparto["20284569875"]
```

La clave es el CUIT y el valor es el nombre. Al consultar con la clave se obtiene el nombre. El CUIT es texto: se usa para identificar, no para hacer una cuenta. En `if cuit in clientesReparto`, la búsqueda se hace entre las claves.

Un bloque es un grupo de instrucciones

```
if numero > 0:
    return numero

print("El número debe ser mayor a cero.")
```

Los espacios al comienzo se llaman sangría. El `return` pertenece al `if` porque está más adentro. El `print` está afuera. Si se ejecuta el `return`, la función termina y ese `print` no se alcanza. Una línea en blanco ayuda a leer, pero no cierra un bloque.

Definir una función y ejecutarla son momentos distintos

Cuando Python lee una definición con `def`, prepara la función. Su cuerpo se ejecuta cuando aparece una llamada con paréntesis. Al llamar a una función, Python va a su cuerpo, trabaja allí y luego vuelve al lugar de la llamada con el resultado, si termina normalmente.

> **Pausa para explicar con tus palabras**
>
> En `detalle.articulo.precio`, empezás en el detalle, seguís su referencia al artículo y finalmente leés el precio de ese artículo. Los puntos son el camino de acceso, no una multiplicación.

PASO 03

3. Qué significa cada palabra del código

Esta sección cubre todos los nombres que aparecen como tokens de Python en los cinco archivos. Las palabras del lenguaje tienen un significado fijo. Los nombres de clases, métodos y variables fueron elegidos para este ejercicio. Mayúsculas y minúsculas importan: `Articulo` y `articulo` son nombres distintos.

Palabras del lenguaje y valores especiales

Palabra o nombre	Explicación
`class`	Palabra de Python que declara una clase, el molde de una familia de objetos.
`def`	Declara una función o un método. Su cuerpo queda preparado para ejecutarse al llamarlo.
`if`	Ejecuta un bloque cuando la condición es verdadera.
`else`	Bloque alternativo de un if; se ejecuta cuando su condición no se cumple.
`for`	Recorre los elementos de una colección y ejecuta un bloque por cada uno.
`in`	En for indica la colección recorrida. En una condición comprueba pertenencia; en un diccionario comprueba claves.
`while`	Repite un bloque mientras su condición sea verdadera.
`return`	Termina una llamada a función y entrega un resultado. Sin return explícito, una función que termina normalmente devuelve None.
`break`	Sale del for o while más cercano que lo contiene. No termina automáticamente la función.
`continue`	Abandona la vuelta actual del bucle más cercano y comienza la siguiente.
`try`	Marca el bloque en el que se intentará una operación que puede lanzar una excepción.
`except`	Define qué hacer cuando el bloque try lanza una excepción del tipo indicado.
`raise`	Lanza una excepción y altera el recorrido normal de la ejecución.
`from`	Introduce el módulo del que se importará un nombre.
`import`	Permite traer y usar un nombre definido en otro módulo.
`not`	Operador lógico que invierte el valor de verdad de una expresión.
`is`	Operador de identidad. Junto con not None comprueba que el resultado no sea el valor especial None.
`True`	Valor booleano verdadero, escrito con mayúscula inicial.
`False`	Valor booleano falso. No es lo mismo que la cadena "False".
`None`	Valor especial que representa ausencia de un resultado; no es cero, lista vacía ni cadena vacía.

Herramientas incorporadas y métodos que usamos

Palabra o nombre	Explicación
`input`	Función incorporada que muestra una pregunta, lee una línea del teclado y devuelve una cadena.
`print`	Función incorporada que escribe una representación en pantalla; devuelve None.
`int`	Tipo entero incorporado y conversor que se usa para pasar de una cadena como "2" al número 2.
`ValueError`	Clase de excepción para un valor inadecuado. int puede lanzarla y el programa también la usa explícitamente.
`staticmethod`	Decorador incorporado que evita recibir automáticamente self en el método decorado.
`append`	Método de lista que agrega un elemento al final y devuelve None. Modifica la lista existente.
`strip`	Método de cadena que devuelve otra cadena sin espacios y otros blancos de ambos extremos.
`upper`	Método de cadena que devuelve una versión en mayúsculas; no borra tildes.

Nombres especiales y clases del trabajo

Palabra o nombre	Explicación
`self`	Nombre convencional del objeto que recibe un método de instancia. No es una palabra reservada.
`__init__`	Método especial que inicializa los datos cuando se construye una instancia. Python lo invoca automáticamente durante esa construcción.
`__name__`	Variable especial del módulo: vale "__main__" cuando se ejecuta ese archivo como programa principal.
`main`	Nombre elegido para el método principal. Python no lo llama por su nombre: hay una llamada explícita al final del archivo.
`Articulo`	Clase que conserva código, denominación y precio. También es el nombre de su módulo en las importaciones.
`RemitoVentaDetalle`	Clase que representa un renglón de venta: cantidad, artículo y subtotal.
`RemitoVenta`	Clase que reúne los detalles vendidos a un cliente en un remito.
`RepartoDiario`	Clase que reúne fecha, lista de remitos y total general.
`CrearRepartoDiario`	Clase que organiza el método de entrada main.

Nombres de funciones y métodos propios

Palabra o nombre	Explicación
`leer_texto`	Función auxiliar que devuelve una cadena no vacía después de recortar sus extremos.
`leer_entero_positivo`	Función auxiliar que reintenta hasta devolver un entero mayor a cero.
`confirmar`	Función auxiliar que reconoce SI/NO y devuelve True o False.
`buscar_articulo`	Función que recorre el catálogo y devuelve el artículo coincidente o None.
`mostrar_reparto`	Función que imprime fecha, remitos, detalles y total. Termina sin return explícito y devuelve None.
`agregar_detalle`	Método que incorpora un detalle y actualiza el total del remito.
`agregar_remito`	Método que valida que el remito tenga detalles, lo incorpora y actualiza el reparto.
`calcular_total`	Nombre de dos métodos: uno suma detalles de un remito y el otro suma importes de un reparto. Se distinguen por el objeto que recibe la llamada.

Nombres de datos y colecciones

Palabra o nombre	Explicación
`codigo`	Identificador de un artículo, almacenado como cadena; por ejemplo, "10".
`denominacion`	Nombre descriptivo del artículo. Se escribió sin tilde como identificador del programa.
`precio`	Importe de una unidad del artículo.
`cantidad`	Número de unidades vendidas en un detalle. La entrada interactiva exige un entero positivo.
`articulo`	Nombre de un parámetro o variable que apunta a un objeto Articulo; también es un atributo del detalle.
`subtotal`	Cantidad multiplicada por precio; se guarda al construir el detalle.
`nombreCliente`	Razón social o nombre que se guarda en el remito.
`numeroRemito`	Número identificador del remito. Se pide como entero positivo.
`detallesRemito`	Lista de objetos RemitoVentaDetalle de un remito.
`totalVenta`	Suma de los subtotales del remito, guardada en un atributo.
`detalle`	Nombre que apunta a un objeto RemitoVentaDetalle recibido, creado o recorrido.
`fecha`	Texto que identifica la fecha del reparto; esta versión comprueba que no esté vacío.
`remitosVenta`	Lista de objetos RemitoVenta pertenecientes al reparto.
`totalReparto`	Suma de los totales de los remitos, almacenada en un atributo.
`remito`	Nombre que apunta a un objeto RemitoVenta.
`mensaje`	Parámetro con la pregunta que mostrará una función de entrada.
`texto`	Variable local que guarda lo leído y recortado en leer_texto.
`numero`	Variable local con el entero convertido; en main se usa para el número de remito.
`respuesta`	Cadena leída, recortada y convertida a mayúsculas para reconocer la respuesta.
`articulosDisponibles`	Lista con los cinco objetos Articulo del catálogo.
`reparto`	Nombre que apunta a un objeto RepartoDiario.
`clientesReparto`	Diccionario que asocia CUIT, como clave de texto, con razón social, como valor de texto.

Palabra o nombre	Explicación
`cuit`	Cadena utilizada para buscar o agregar un cliente en el diccionario.

PASO 04

4. Qué significa cada símbolo

El contexto cambia el significado de algunos signos. El punto de `articulo.precio` no es el punto decimal de `0.0`. Las llaves de un diccionario no cumplen la misma función que las de una cadena con `f`.

Símbolo	Uso y explicación
=	Asignación. Guarda o vincula el resultado de la derecha con el destino de la izquierda. Ejemplo: numero = 2.
==	Comparación. Pregunta si dos valores son iguales. Devuelve True o False y no asigna.
<=	Comparación. Pregunta si el valor de la izquierda es menor o igual que el de la derecha.
>	Comparación. Fuera de una cadena de formato, compara si un número es mayor que otro.
+=	Acumulación. En estos números suma el nuevo importe al valor anterior y guarda el resultado.
*	Multiplicación o repetición. cantidad * precio multiplica números. "-" * 70 repite una cadena setenta veces.
.	Acceso o punto decimal. articulo.precio accede a un atributo. .append llama un método. En 0.0 el punto es decimal.
()	Paréntesis. Declaran parámetros, reúnen argumentos de una llamada, agrupan expresiones o delimitan una tupla con comas.
,	Coma. Separa parámetros, argumentos o elementos de una colección. Puede aparecer al final de una lista o diccionario.
:	Dos puntos. Abre un bloque; separa clave y valor en un diccionario; dentro de una cadena con f inicia una regla de formato.
[]	Corchetes. [] crea una lista vacía. [elementos] crea una lista con contenido. diccionario[clave] accede a un valor.
{}	Llaves. Con pares clave: valor crean un diccionario. Dentro de una cadena con f delimitan una expresión a reemplazar.
"..." / '...'	Comillas. Delimitan cadenas de texto; las comillas delimitadoras no se imprimen. Permiten distinguir "10" del número 10.
#	Comentario. Fuera de una cadena, indica un comentario hasta el final de esa línea.
@	Decorador. En @staticmethod aplica esa característica al método definido inmediatamente después.

Símbolo	Uso y explicación
_	Guion bajo. Forma parte de nombres como leer_texto; no separa instrucciones. Los nombres especiales usan dobles guiones bajos.
\n	Salto de línea. Dentro de una cadena normal produce un salto de línea. Es distinto de /n.
f"..."	Cadena formateada. Evalúa expresiones entre llaves y coloca su representación dentro del texto.
$ / - / SI/NO	Texto literal. Dentro de comillas son caracteres del mensaje. Un guion allí no es una resta ni una barra es una división.
:.2f	Formato numérico. Muestra dos decimales en notación fija. No cambia el dato almacenado.
:>8 / :<30	Alineación. Alinea a derecha o izquierda con un ancho mínimo. Si el contenido es más largo, no se recorta.
:>15.2f	Formato combinado. Alinea a la derecha con ancho mínimo 15 y muestra dos decimales.
Sangría	Estructura. Los espacios iniciales determinan qué instrucciones pertenecen a cada clase, función, condición o bucle.

Desarmamos una cadena con formato

```
f"${detalle.subtotal:>12.2f}"
```

Parte	Lectura
f antes de las comillas	Este texto puede reemplazar expresiones entre llaves.
$	Es un carácter que se imprime tal cual.
detalle.subtotal	Se obtiene el número guardado en el detalle.
:	Empieza la regla para mostrar ese número.
>	Alinear el contenido a la derecha.
12	Usar un ancho mínimo de doce caracteres.
.2	Mostrar dos cifras después del punto decimal.
f dentro del formato	Representar el número en notación de punto fijo.

Si el subtotal es 20000, se presenta como 20000.00, con espacios de relleno para la alineación. El valor del atributo sigue siendo el mismo. El ancho es mínimo: un dato más largo no se corta automáticamente.

Por qué una instrucción ocupa varias líneas

```
cantidad = leer_entero_positivo(
    "Cantidad: "
)
```

Los paréntesis abiertos permiten continuar una instrucción en varias líneas. Aquí hay una sola llamada, con un solo argumento. La línea que cierra `)` es parte de la misma instrucción. Lo mismo ocurre con las listas abiertas por `[` y los diccionarios abiertos por `{`.

PASO 05

5. El programa, línea por línea

Los números de esta sección coinciden con tus archivos actuales. Cada archivo empieza nuevamente en L001. Los números son guías de lectura: no forman parte del código. También se explican las líneas en blanco y las que solo cierran paréntesis.

Leé una ficha, mirá la siguiente y anticipá el recorrido antes de avanzar. La versión completa de cada archivo está reunida al final para ver de nuevo la sangría y los bloques.

Articulo.py

Este es el molde más pequeño. Solo conserva los datos del catálogo. Leelo como una ficha de producto: código, nombre y precio.

5 líneas en este archivo. La numeración incluye las líneas en blanco.

L001 a L005 · El molde del producto

```
L001 | class Articulo:
```

`class` declara una clase: el molde para crear artículos. `Articulo` es el nombre elegido para ese molde. Los dos puntos anuncian que abajo empieza su contenido, escrito con sangría.

```
L002 |     def __init__(self, codigo, denominacion, precio):
```

`def` define un método. `__init__` inicializa cada artículo nuevo. `self` representa ese artículo. `codigo`, `denominacion` y `precio` son parámetros: datos que recibirá al crearlo. Los paréntesis reúnen los parámetros y las comas los separan.

```
L003 |         self.codigo = codigo
```

Guarda el parámetro `codigo` dentro del artículo actual. A la izquierda, `self.codigo` es el atributo que conservará el dato. A la derecha, `codigo` es el dato recibido. El signo = asigna; no compara.

```
L004 |         self.denominacion = denominacion
```

Guarda la descripción recibida en el atributo `denominacion`. Por ejemplo, el objeto puede conservar el texto Filtro de Aceite. El punto permite acceder a una parte del objeto.

```
L005 |         self.precio = precio
```

Guarda el precio recibido. Si se pasa 10000, ese artículo conservará 10000 en `self.precio`. Esta clase almacena datos; el cálculo de una venta se hará en el detalle.

RemitoVentaDetalle.py

Un detalle es un renglón de venta. Une un artículo del catálogo con una cantidad y calcula el importe de ese renglón.

8 líneas en este archivo. La numeración incluye las líneas en blanco.

L001 a L008 · Cantidad, artículo y subtotal

```
L001 | class RemitoVentaDetalle:
```

Declara el molde de un renglón del remito. Un detalle reúne una cantidad, un artículo y un subtotal. Dos artículos vendidos en renglones separados producen dos objetos de esta clase.

```
L002 |     def __init__(self, cantidad, articulo):
```

Define el inicializador. Recibe `cantidad` y un objeto `articulo` ya existente. `self` será el detalle que se está creando. Aquí no se vuelve a construir el artículo.

```
L003 |         if cantidad <= 0:
```

`if` pregunta si la cantidad es menor o igual que cero. <= es una comparación: devuelve verdadero o falso. Si es verdadero, Python entra en el bloque que tiene más sangría.

```
L004 |             raise ValueError("La cantidad debe ser mayor a cero.")
```

`raise` lanza una excepción. `ValueError` indica que el valor recibido no sirve. El texto entre comillas explica la causa. La creación se interrumpe: las asignaciones de las líneas 6 a 8 no llegan a ejecutarse.

L005 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L006 |         self.cantidad = cantidad
```

Conserva la cantidad válida en este detalle. Por ejemplo, `self.cantidad` queda en 2. La validación por teclado que veremos después admite enteros positivos.

```
L007 |         self.articulo = articulo
```

Guarda una referencia al objeto artículo recibido. Permite leer después `detalle.articulo.codigo` o `detalle.articulo.precio`. No crea una copia independiente del artículo.

```
L008 |         self.subtotal = cantidad * articulo.precio
```

* multiplica la cantidad por el precio del objeto artículo. Guarda el resultado en `subtotal`. Con cantidad 2 y precio 10000, el subtotal queda en 20000. Se calcula en este momento de la creación.

RemitoVenta.py

Un remito es una hoja de venta para un cliente. Contiene varios renglones y sabe sumar sus importes.

18 líneas en este archivo. La numeración incluye las líneas en blanco.

L001 a L007 · Crear un remito vacío

```
L001 | class RemitoVenta:
```

Declara la clase que representa un remito completo: cliente, número, renglones de venta y total.

```
L002 |     def __init__(self, nombreCliente, numeroRemito):
```

Define el inicializador. Recibe nombre de cliente y número de remito. `self` representa el remito nuevo que conservará esos datos.

```
L003 |         self.nombreCliente = nombreCliente
```

Guarda el nombre recibido en el atributo `nombreCliente` del remito actual.

```
L004 |         self.numeroRemito = numeroRemito
```

Guarda el número recibido en `numeroRemito`. En la carga por teclado se pide como entero positivo.

```
L005 |         self.detallesRemito = []
```

`[]` crea una lista vacía para los detalles. Como esta instrucción está dentro de `__init__`, cada remito recibe su propia lista. Los detalles de otro remito no se mezclan con esta.

```
L006 |         self.totalVenta = 0.0
```

Inicializa el total de venta en cero decimal. `0.0` es un número de tipo `float`. Todavía no hay detalles que sumar.

L007 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L008 a L011 · Agregar un detalle

```
L008 |     def agregar_detalle(self, detalle):
```

Define el método `agregar_detalle`. Recibe el remito actual mediante `self` y el detalle que se quiere incorporar mediante el parámetro `detalle`.

```
L009 |         self.detallesRemito.append(detalle)
```

Accede a la lista del remito y llama a `append`, que agrega un elemento al final. Lo que se agrega es el objeto `detalle`, no solamente su subtotal. `append` modifica esa lista.

```
L010 |         self.calcular_total()
```

Llama a otro método del mismo remito para actualizar su total. Los paréntesis vacíos significan que no enviamos argumentos explícitos; Python ya aporta `self` al método de instancia.

L011 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L012 a L018 · Sumar los subtotales

```
L012 |     def calcular_total(self):
```

Define el método que calcula el total de este remito. Definirlo describe qué hará; su cuerpo se ejecuta cuando alguien lo llama, por ejemplo en la línea 10.

```
L013 |         self.totalVenta = 0.0
```

Reinicia el acumulador en cero antes de recorrer los detalles. Este reinicio evita sumar sobre el total de un cálculo anterior.

L014 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L015 |         for detalle in self.detallesRemito:
```

`for` recorre la lista. En cada vuelta, el nombre temporal `detalle` apunta al siguiente objeto. `in` indica qué colección se recorre. Las instrucciones con más sangría se repiten para cada elemento.

```
L016 |             self.totalVenta += detalle.subtotal
```

`+=` suma y vuelve a guardar. Equivale a escribir `self.totalVenta = self.totalVenta + detalle.subtotal`. Con subtotales 20000 y 8000, el acumulador pasa por 0, 20000 y 28000.

L017 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L018 |         return self.totalVenta
```

`return` entrega el total a quien llamó al método y termina esa llamada. El valor también queda guardado en el atributo `totalVenta`. No se imprime nada por usar `return`.

RepartoDiario.py

El reparto es la carpeta del día. Guarda las hojas de venta y suma el importe de todos los remitos.

22 líneas en este archivo. La numeración incluye las líneas en blanco.

L001 a L006 · Crear la carpeta del día

```
L001 | class RepartoDiario:
```

Declara la clase que representa el reparto de una fecha y contiene sus remitos de venta.

```
L002 |     def __init__(self, fecha):
```

Define el inicializador. Recibe la fecha como texto y trabaja sobre el reparto nuevo identificado por `self`.

```
L003 |         self.fecha = fecha
```

Conserva la fecha recibida. Por ejemplo, el texto 03/11/2026. Guardar este texto no comprueba que sea una fecha real.

```
L004 |         self.remitosVenta = []
```

Crea una lista vacía para los remitos del reparto. Cada instancia tendrá su propia lista.

```
L005 |         self.totalReparto = 0.0
```

Empieza el monto total del reparto en cero decimal. Todavía no se agregó ningún remito.

L006 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L007 a L015 · Validar e incorporar un remito

```
L007 |     def agregar_remito(self, remito):
```

Define el método que recibe un objeto `remito` para incorporarlo al reparto actual.

```
L008 |         if not remito.detallesRemito:
```

Una lista vacía se evalúa como falsa. `not` invierte ese valor: la condición es verdadera cuando el remito no tiene detalles. No modifica ni vacía la lista.

```
L009 |             raise ValueError(
```

Empieza a lanzar `ValueError`. Se abre el paréntesis y el mensaje se escribe en la siguiente línea para mejorar la lectura. Es una sola instrucción distribuida en varias líneas.

```
L010 |                 "El remito debe tener al menos un detalle."
```

Este texto es el argumento de la excepción: explica que hace falta un detalle. Por sí solo no se imprime; forma parte de la excepción que se está construyendo.

```
L011 |             )
```

Cierra el paréntesis abierto en la línea 9. Si se había entrado en el `if`, la excepción interrumpe el método y el remito no se agrega.

L012 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L013 |         self.remitosVenta.append(remito)
```

Agrega el objeto remito al final de la lista del reparto. Esta línea se alcanza cuando la validación anterior no lanzó una excepción.

```
L014 |         self.calcular_total()
```

Recalcula el total del reparto después de incorporar el remito. Así, el atributo `totalReparto` queda actualizado.

L015 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L016 a L022 · Sumar los remitos

```
L016 |     def calcular_total(self):
```

Define el método que sumará los importes de todos los remitos del reparto.

```
L017 |         self.totalReparto = 0.0
```

Reinicia el acumulador antes de sumar. Recalcular dos veces con los mismos datos produce el mismo total y no lo duplica.

L018 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L019 |         for remito in self.remitosVenta:
```

Recorre la lista de remitos. En cada vuelta, `remito` representa uno de sus objetos.

```
L020 |             self.totalReparto += remito.calcular_total()
```

Primero se ejecuta `remito.calcular_total()` y se recibe su resultado. Después, `+=` agrega ese resultado al acumulador del reparto. Se reutiliza el cálculo que ya sabe hacer cada remito.

L021 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L022 |         return self.totalReparto
```

Devuelve el total del reparto a quien hizo la llamada. Por ejemplo, si los remitos valen 28000 y 22500, devuelve 50500.

CrearRepartoDiario.py

Este archivo coordina el recorrido: pide datos, usa las clases y muestra el resultado. Primero define las herramientas; al final llama a main.

190 líneas en este archivo. La numeración incluye las líneas en blanco.

L001 a L006 · Importar las clases

```
L001 | from Articulo import Articulo
```

`from` indica el módulo de origen e `import` trae un nombre de ese módulo. El primer `Articulo` se refiere al archivo Articulo.py; el segundo, a la clase definida dentro. Importar permite usar esa clase aquí.

```
L002 | from RemitoVentaDetalle import RemitoVentaDetalle
```

Trae la clase `RemitoVentaDetalle` desde su archivo. Importar la clase todavía no crea ningún detalle de venta.

```
L003 | from RemitoVenta import RemitoVenta
```

Trae la clase `RemitoVenta` para poder construir remitos en el programa principal.

```
L004 | from RepartoDiario import RepartoDiario
```

Trae la clase `RepartoDiario` para poder crear el reparto que reunirá los remitos.

L005 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L006 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L007 a L016 · leer_texto: pedir un texto útil

```
L007 | def leer_texto(mensaje):
```

Define una función auxiliar llamada `leer_texto`. Recibe `mensaje`, el texto que mostrará para pedir un dato. Es una función del módulo, por eso no tiene parámetro `self`.

```
L008 |     while True:
```

`while` repite mientras su condición sea verdadera. `True` siempre es verdadero: este bucle necesita una salida interna. Aquí saldrá con `return` cuando el texto sea válido.

```
L009 |         texto = input(mensaje).strip()
```

`input(mensaje)` muestra la pregunta, espera lo escrito y lo devuelve como cadena. `.strip()` quita espacios y saltos de línea de ambos extremos. `texto` conserva el resultado; los espacios internos se mantienen.

L010 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L011 |         if texto:
```

`if texto` pregunta si la cadena tiene contenido después de recortar los extremos. La cadena vacía se considera falsa. Por ejemplo, escribir solo espacios termina produciendo una cadena vacía.

```
L012 |             return texto
```

Si el texto tiene contenido, se devuelve y termina toda esta llamada a `leer_texto`, incluyendo su bucle. El programa vuelve al lugar donde pidió el dato.

L013 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L014 |         print("El texto no puede quedar vacío.")
```

Muestra el aviso si no se ejecutó el `return`. Al llegar al final de la vuelta, el `while` vuelve a pedir el dato. `print` muestra texto en pantalla.

L015 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L016 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L017 a L030 · leer_entero_positivo: convertir y validar

```
L017 | def leer_entero_positivo(mensaje):
```

Define la función que pedirá un entero mayor a cero. El parámetro `mensaje` permite reutilizarla para número de remito o cantidad.

```
L018 |     while True:
```

Empieza el bucle de reintentos. Seguirá pidiendo datos hasta que un `return` entregue un entero positivo.

```
L019 |         try:
```

`try` marca una operación que podría fallar. Python intentará convertir la entrada en un número entero.

```
L020 |             numero = int(input(mensaje))
```

Primero `input` obtiene una cadena. Después `int` intenta convertirla en entero y, si puede, se asigna a `numero`. El texto "2" se convierte en el número 2; "hola" y "2.5" no se convierten con esta instrucción.

```
L021 |         except ValueError:
```

`except ValueError` recibe el control si falló la conversión por un contenido inválido. No es el camino de una entrada correcta.

```
L022 |             print("Ingresá un número entero.")
```

Muestra el aviso de conversión inválida. Por ejemplo, se ejecuta cuando se ingresó el texto "hola".

```
L023 |             continue
```

`continue` termina esta vuelta del `while` y empieza otra. Por eso no se compara `numero` en esa vuelta: se vuelve directamente a pedir un dato.

L024 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L025 |         if numero > 0:
```

Cuando la conversión funcionó, comprueba si el entero es mayor a cero. `>` compara; no modifica el número.

```
L026 |             return numero
```

Entrega el entero válido y termina la función. Quien llamó recibe un número, no la cadena original de `input`.

L027 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L028 |         print("El número debe ser mayor a cero.")
```

Si el número era cero o negativo, muestra este aviso. La vuelta termina y el `while` vuelve a pedirlo. Es un problema distinto de escribir letras: aquí la conversión sí había funcionado.

L029 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L030 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L031 a L045 · confirmar: transformar SI/NO en un booleano

```
L031 | def confirmar(mensaje):
```

Define la función `confirmar`, que convertirá respuestas del usuario en los valores booleanos `True` o `False`.

```
L032 |     while True:
```

Repite la pregunta hasta recibir una respuesta reconocida. Escribir algo distinto de SI o NO no termina la función.

```
L033 |         respuesta = input(
```

Empieza una llamada a `input` repartida en tres líneas. El resultado final se guardará en `respuesta` después de recortar espacios y convertir a mayúsculas.

```
L034 |             f"{mensaje} (SI/NO): "
```

La letra `f` antes de las comillas crea una cadena formateada. `{mensaje}` se reemplaza por el contenido del parámetro; el texto (SI/NO) se agrega tal cual. Las barras y los paréntesis dentro de las comillas son parte del mensaje.

```
L035 |         ).strip().upper()
```

Cierra `input(...)`, aplica `.strip()` al texto recibido y luego `.upper()`. Por ejemplo, " si " termina como "SI". El orden es: pedir, recortar, pasar a mayúsculas y asignar.

L036 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L037 |         if respuesta in ("SI", "SÍ", "S"):
```

`in` pregunta si `respuesta` pertenece a la tupla de opciones afirmativas. Las comas separan sus elementos. Se aceptan SI, SÍ y S.

```
L038 |             return True
```

Devuelve el booleano `True`, que significa respuesta afirmativa. Es un valor lógico, diferente de la cadena "True".

L039 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L040 |         if respuesta in ("NO", "N"):
```

Comprueba si la respuesta pertenece a las opciones negativas NO o N.

```
L041 |             return False
```

Devuelve el booleano `False`, que significa respuesta negativa, y termina la función.

L042 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L043 |         print("Respondé SI o NO.")
```

Si ninguna condición anterior devolvió un resultado, muestra el aviso y se repite la pregunta.

L044 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L045 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L046 a L053 · buscar_articulo: encontrar el objeto del catálogo

```
L046 | def buscar_articulo(codigo, articulosDisponibles):
```

Define una búsqueda que recibe el código buscado y la lista donde debe buscar. Su resultado será un objeto artículo o el valor `None`.

```
L047 |     for articulo in articulosDisponibles:
```

Recorre los artículos disponibles uno por uno. El nombre `articulo` apunta al elemento de la vuelta actual.

```
L048 |         if articulo.codigo == codigo:
```

`==` compara el código del artículo actual con el código recibido. Ambos se manejan como cadenas, por ejemplo `"10"`.

```
L049 |             return articulo
```

Si los códigos coinciden, devuelve ese mismo objeto artículo y termina la función inmediatamente. No hace falta seguir recorriendo los otros artículos.

L050 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L051 |     return None
```

Se alcanza después de terminar todo el `for` sin coincidencias. Devuelve `None`, que aquí significa "no se encontró". La sangría es importante: esta línea está fuera del `for`.

L052 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L053 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L054 a L064 · mostrar_reparto: comenzar la presentación

```
L054 | def mostrar_reparto(reparto):
```

Define la función de presentación. Recibe un objeto reparto y lo utiliza para imprimir su información.

```
L055 |     reparto.calcular_total()
```

Actualiza el total antes de mostrarlo. El resultado queda en el atributo del objeto aunque esta llamada no lo asigne a una variable local.

L056 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L057 |     print("\nReparto Diario")
```

Imprime el título. La secuencia `\n` dentro de la cadena produce un salto de línea; no imprime los caracteres barra y letra n.

```
L058 |     print(f"Fecha: {reparto.fecha}")
```

La cadena con `f` coloca el valor de `reparto.fecha` dentro del mensaje. El punto accede al atributo del objeto.

```
L059 |     print("\n-------------- Remitos del Reparto --------------")
```

Imprime un separador visual antes de los remitos. Los guiones que están dentro de las comillas son texto, no restas.

L060 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L061 |     for remito in reparto.remitosVenta:
```

Recorre los remitos del reparto. Todo lo que mantiene una sangría adicional debajo se hará una vez por remito.

```
L062 |         print(f"\nNombre Cliente: {remito.nombreCliente}")
```

Muestra el nombre del cliente del remito actual. El salto de línea ayuda a separar un remito del anterior.

```
L063 |         print(f"Número Remito: {remito.numeroRemito}")
```

Muestra el número del remito actual usando su atributo `numeroRemito`.

L064 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L065 a L085 · mostrar_reparto: columnas, detalles y totales

```
L065 |         print(
```

Abre una llamada a `print` cuyo mensaje se forma con varias cadenas escritas en las líneas siguientes. Sigue siendo una sola llamada.

```
L066 |             f"{'Cantidad':>8} "
```

Forma la primera columna del encabezado. En una cadena con `f`, `:>8` pide alinear a la derecha en un ancho mínimo de 8 caracteres. El texto Cantidad está entre comillas internas porque es una cadena literal.

```
L067 |             f"{'Denominación Artículo':<30} "
```

Forma la columna Denominación Artículo. `:<30` alinea a la izquierda con ancho mínimo 30. El signo < aquí es una regla de formato dentro de la cadena, no una comparación de números.

```
L068 |             f"{'Precio Unitario':>15} "
```

Forma la columna Precio Unitario con ancho mínimo 15 y alineación a la derecha mediante `:>15`.

```
L069 |             f"{'Subtotal':>12}"
```

Forma la columna Subtotal con ancho mínimo 12 y alineación a la derecha. Las cadenas consecutivas dentro de estos paréntesis se unen en un mismo mensaje.

```
L070 |         )
```

Cierra la única llamada a `print` que empezó en la línea 65. En ese momento se muestra el encabezado completo.

L071 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L072 |         for detalle in remito.detallesRemito:
```

Dentro del remito actual, recorre sus detalles. Es un bucle anidado: por cada remito se recorren sus renglones de venta.

```
L073 |             print(
```

Empieza una llamada a `print` para mostrar una fila de detalle. Los valores de sus columnas se escriben a continuación.

```
L074 |                 f"{detalle.cantidad:>8} "
```

Coloca la cantidad del detalle en una columna alineada a la derecha, con ancho mínimo de 8 caracteres.

```
L075 |                 f"{detalle.articulo.denominacion:<30} "
```

Sigue dos referencias: del detalle a su artículo y del artículo a su denominación. Luego alinea ese texto a la izquierda con ancho mínimo 30.

```
L076 |                 f"{detalle.articulo.precio:>15.2f} "
```

Muestra el precio del artículo. `:>15.2f` significa ancho mínimo 15, alineación derecha y dos cifras decimales en formato fijo. Cambia la presentación, no el precio guardado.

```
L077 |                 f"{detalle.subtotal:>12.2f}"
```

Muestra el subtotal ya calculado, con ancho mínimo 12 y dos decimales. `f` dentro de la regla de formato indica formato numérico fijo; la `f` anterior a las comillas indica cadena formateada.

```
L078 |             )
```

Cierra la llamada que imprime la fila. El `for` seguirá con el próximo detalle, si existe.

L079 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L080 |         print(f"Total Remito Venta: ${remito.totalVenta:.2f}")
```

Fuera del bucle de detalles, pero dentro del de remitos, muestra una vez el total del remito. `$` es texto literal y `:.2f` pide dos decimales.

```
L081 |         print("-" * 70)
```

Repite la cadena "-" setenta veces y la imprime. * entre una cadena y un entero repite texto; entre dos números, como en el subtotal, multiplica.

L082 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L083 |     print(f"\nMonto Total Reparto: ${reparto.totalReparto:.2f}")
```

Por su menor sangría está fuera de ambos bucles. Muestra una sola vez el total general, después de todos los remitos.

L084 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L085 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L086 a L096 · main: crear el catálogo

```
L086 | class CrearRepartoDiario:
```

Declara la clase encargada de organizar la carga. Esta clase cumple el papel de entrada principal pedido por el enunciado.

```
L087 |     @staticmethod
```

`@staticmethod` es un decorador: indica que el siguiente método puede llamarse desde la clase sin recibir un objeto como `self`. En este caso organiza variables locales y no necesita atributos de una instancia de carga.

```
L088 |     def main():
```

Define el método `main` sin parámetros. El nombre main es una convención del programador, no una palabra reservada que Python ejecute automáticamente. La llamada de la línea 190 lo pondrá en marcha.

```
L089 |         articulosDisponibles = [
```

Empieza una lista llamada `articulosDisponibles`. El signo = asigna la lista; el corchete `[` abre su contenido.

```
L090 |             Articulo("10", "Filtro de Aceite", 10000),
```

Construye un artículo con código "10", denominación Filtro de Aceite y precio 10000. La posición de los argumentos coincide con los parámetros de `Articulo.__init__`. La coma lo separa del siguiente elemento de la lista.

```
L091 |             Articulo("20", "Filtro de Aire", 8000),
```

Construye y agrega a la lista el artículo "20", Filtro de Aire, con precio 8000. Es otro objeto, aunque usa el mismo molde `Articulo`.

```
L092 |             Articulo("30", "Filtro de Combustible", 7500),
```

Construye el artículo "30", Filtro de Combustible, con precio 7500.

```
L093 |             Articulo("40", "Aceite de Motor sw/10", 25000),
```

Construye el artículo "40", Aceite de Motor sw/10, con precio 25000. La barra dentro de la denominación es parte del texto; no se ejecuta una división.

```
L094 |             Articulo("50", "Correa de Distribución", 20000),
```

Construye el artículo "50", Correa de Distribución, con precio 20000. La coma final está permitida y no crea un sexto elemento vacío.

```
L095 |         ]
```

Cierra la lista abierta en la línea 89. Ahora contiene cinco referencias a objetos `Articulo`.

L096 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L097 a L107 · main: clientes, fecha y reparto

```
L097 |         clientesReparto = {
```

Empieza un diccionario llamado `clientesReparto`. Las llaves aquí crean una colección de pares clave y valor. No cumplen el mismo papel que las llaves dentro de una cadena con `f`.

```
L098 |             "20284569875": "Juan Alonso",
```

Registra un par: la clave es el CUIT "20284569875" y el valor es "Juan Alonso". Los dos puntos separan clave y valor; la coma separa este par del siguiente.

```
L099 |             "20157896542": "Emiliano Salas",
```

Registra el CUIT "20157896542" asociado con Emiliano Salas.

```
L100 |             "50258963654": "Pacallcar Autos",
```

Registra el CUIT "50258963654" asociado con Pacallcar Autos.

```
L101 |             "27654987456": "Julieta Videla",
```

Registra el CUIT "27654987456" asociado con Julieta Videla.

```
L102 |             "50335588945": "Todo Auto SRL",
```

Registra el CUIT "50335588945" asociado con Todo Auto SRL. La coma final también es válida en un diccionario.

```
L103 |         }
```

Cierra el diccionario. Hay cinco claves y cada una permite recuperar su razón social.

L104 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L105 |         fecha = leer_texto("Fecha del reparto: ")
```

Llama a `leer_texto` para pedir la fecha y guarda su resultado en la variable local `fecha`. La función terminará antes de continuar con la línea siguiente.

```
L106 |         reparto = RepartoDiario(fecha)
```

Construye un objeto `RepartoDiario` usando la fecha recibida. La variable `reparto` permite acceder a ese objeto, que comienza con lista vacía y total cero.

L107 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L108 a L128 · main: empezar un remito y resolver el cliente

```
L108 |         # Bucle de carga de remitos.
```

Es un comentario: el símbolo # hace que Python ignore el texto que lo sigue en esta línea. Sirve como explicación para la persona que lee el programa.

```
L109 |         while True:
```

Empieza el bucle exterior de carga de remitos. Se saldrá con el `break` de la línea 184 cuando el usuario no quiera otro remito.

```
L110 |            print("\nNUEVO REMITO")
```

Muestra el título de una nueva carga de remito al comienzo de cada vuelta del bucle exterior.

L111 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L112 |            numero = leer_entero_positivo(
```

Empieza una llamada a `leer_entero_positivo`. El resultado se guardará en `numero` después de completar la llamada.

```
L113 |                "Número de remito: "
```

Este texto será el argumento `mensaje` de la función: le dice al usuario qué debe ingresar.

```
L114 |            )
```

Cierra la llamada. Cuando la función devuelve un entero válido, ese entero se asigna a `numero` y continúa la carga.

L115 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L116 |            cuit = leer_texto("CUIT del cliente: ")
```

Pide el CUIT y lo guarda como cadena en `cuit`. No se convierte a entero porque se usa como identificador y como clave del diccionario.

L117 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L118 |            if cuit in clientesReparto:
```

`in` aplicado a un diccionario busca entre sus claves. Esta condición pregunta si el CUIT ya está registrado, no si aparece entre las razones sociales.

```
L119 |                nombreCliente = clientesReparto[cuit]
```

Usa el CUIT como clave entre corchetes para recuperar su razón social. Guarda el texto obtenido en `nombreCliente`.

```
L120 |                print(f"Cliente: {nombreCliente}")
```

Muestra el nombre encontrado. Esta instrucción pertenece al camino del cliente que ya existía.

```
L121 |            else:
```

`else` abre el camino alternativo: se ejecuta cuando el CUIT no estaba entre las claves del diccionario.

```
L122 |                nombreCliente = leer_texto(
```

Empieza una llamada a `leer_texto` para pedir el nombre del cliente nuevo. El resultado se guardará en `nombreCliente`.

```
L123 |                    "Cliente nuevo. Ingresá su razón social: "
```

Es el mensaje enviado a la función auxiliar. Explica que se necesita la razón social del cliente que no estaba registrado.

```
L124 |                 )
```

Cierra la llamada. Después de obtener un nombre no vacío, ese nombre queda en `nombreCliente`.

```
L125 |                 clientesReparto[cuit] = nombreCliente
```

Agrega al diccionario la asociación entre el CUIT nuevo y su nombre. Si ese CUIT vuelve a ingresarse durante esta misma ejecución, ya se encontrará en el camino del `if`.

L126 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L127 |             remito = RemitoVenta(nombreCliente, numero)
```

Construye un remito usando el cliente resuelto por cualquiera de los dos caminos y el número recibido. Este remito nuevo empieza sin detalles y con total cero.

L128 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L129 a L141 · main: mostrar artículos y empezar los detalles

```
L129 |             print("\nARTÍCULOS DISPONIBLES")
```

Imprime el título del catálogo que se mostrará antes de cargar los detalles de este remito.

L130 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L131 |             for articulo in articulosDisponibles:
```

Recorre los cinco artículos del catálogo para mostrarlos. El nombre de vuelta `articulo` se reutilizará después para el artículo elegido.

```
L132 |                 print(
```

Empieza una llamada a `print` para mostrar un artículo del catálogo en una sola línea.

```
L133 |                     f"{articulo.codigo} - "
```

Agrega el código del artículo y un guion de separación al mensaje que se está formando.

```
L134 |                     f"{articulo.denominacion} - "
```

Agrega la denominación del artículo y otro guion de separación. La cadena se une con las otras de esta llamada.

```
L135 |                     f"${articulo.precio:.2f}"
```

Agrega el precio con signo monetario y dos decimales. Por ejemplo, el número 10000 se muestra como $10000.00.

```
L136 |                 )
```

Cierra el `print` del artículo actual. Si quedan artículos, el `for` continúa con el siguiente.

L137 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L138 |             # Bucle de carga de detalles.
```

Comentario que identifica el comienzo de la carga de renglones del remito.

```
L139 |             # La pregunta de salida aparece después del primer detalle.
```

Comentario que recuerda una regla clave: se pregunta por otro detalle después de cargar el primero. Eso asegura al menos un detalle en la carga normal.

```
L140 |             while True:
```

Empieza el bucle intermedio, que repite detalles del remito actual. El `break` de la línea 177 terminará este bucle cuando se responda NO a otro detalle.

L141 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L142 a L158 · main: repetir hasta encontrar un código

```
L142 |                 # Repetir la búsqueda hasta ingresar un código válido.
```

Comentario que explica por qué habrá un tercer bucle: un código inválido debe volver a pedirse sin abandonar el detalle actual.

```
L143 |                 while True:
```

Empieza el bucle interior de búsqueda de artículo. Se repite hasta encontrar un código del catálogo.

```
L144 |                     codigo = leer_texto(
```

Empieza a llamar a `leer_texto` para pedir el código. El resultado se asignará a `codigo`.

```
L145 |                         "Código del artículo: "
```

Es el mensaje que se verá en pantalla para ingresar el identificador del artículo.

```
L146 |                     )
```

Cierra la llamada. Queda guardada una cadena no vacía, por ejemplo "10". Todavía falta comprobar que exista en el catálogo.

L147 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L148 |                     articulo = buscar_articulo(
```

Empieza la llamada a `buscar_articulo`. Su resultado se guardará en `articulo`.

```
L149 |                         codigo, articulosDisponibles
```

Envía dos argumentos en orden: el código escrito y la lista del catálogo. La coma los separa.

```
L150 |                     )
```

Cierra la llamada y asigna el resultado: un objeto `Articulo` encontrado o None si no hubo coincidencias.

L151 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L152 |                     if articulo is not None:
```

`is not` None comprueba que el resultado no sea el valor especial None. Aquí significa que se encontró un artículo y se puede continuar.

```
L153 |                         break
```

`break` sale del bucle más cercano: el de búsqueda iniciado en la línea 143. No termina la carga de detalles, ni la de remitos, ni la función `main`.

L154 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L155 |                     print(
```

Empieza la impresión del aviso de código inválido. Solo se alcanza si no se ejecutó el `break` anterior.

```
L156 |                         "el código es incorrecto, ingrese nuevamente"
```

Es el mensaje pedido por el enunciado para un artículo que no está en el catálogo.

```
L157 |                     )
```

Cierra el aviso. Al terminar esta vuelta, el `while` interior vuelve a pedir un código.

L158 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L159 a L173 · main: construir e incorporar el detalle

```
L159 |                 cantidad = leer_entero_positivo(
```

Después de salir de la búsqueda, pide una cantidad mediante la función que acepta enteros positivos.

```
L160 |                     "Cantidad: "
```

Es el mensaje de la pregunta. La validación está dentro de `leer_entero_positivo`, por eso no se repite aquí su código.

```
L161 |                 )
```

Cierra la llamada y guarda el entero recibido en `cantidad`. Cero, negativos o texto no llegan como resultado válido.

L162 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L163 |                 detalle = RemitoVentaDetalle(
```

Empieza la construcción del objeto detalle de venta. La variable `detalle` guardará una referencia al resultado.

```
L164 |                     cantidad, articulo
```

Entrega la cantidad validada y el artículo encontrado. El inicializador del detalle conservará ambos y calculará su subtotal.

```
L165 |                 )
```

Cierra la construcción. Ahora existe un detalle con cantidad, referencia a un artículo y subtotal calculado.

L166 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L167 |                 remito.agregar_detalle(detalle)
```

Incorpora el detalle al remito actual. El método agrega a la lista y recalcula `totalVenta`, como vimos en RemitoVenta.py.

L168 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L169 |                 print(
```

Empieza una llamada a `print` para confirmar en pantalla el importe del detalle recién cargado.

```
L170 |                     f"Subtotal del detalle: "
```

Agrega el texto Subtotal del detalle al mensaje. Esta cadena lleva `f`, aunque no contiene campos de reemplazo; funciona igualmente.

```
L171 |                     f"${detalle.subtotal:.2f}"
```

Agrega el valor del subtotal con signo monetario y dos decimales. Se consulta el dato ya guardado en el objeto.

```
L172 |                 )
```

Cierra la impresión de confirmación. A continuación se preguntará si se desea otro detalle.

L173 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L174 a L185 · main: decidir qué seguir cargando

```
L174 |                 if not confirmar(
```

Empieza una condición que usa la respuesta de `confirmar`. `not` invertirá el booleano devuelto para preguntar si la respuesta fue negativa.

```
L175 |                     "¿Querés agregar otro detalle?"
```

Es el mensaje que se envía a `confirmar`. SI devolverá True; NO devolverá False.

```
L176 |                 ):
```

Cierra la llamada y completa la condición. Con NO: `not False` es True y se ejecutará el bloque del `if`. Con SI: `not True` es False y no se entra en ese bloque.

```
L177 |                     break
```

Sale del bucle más cercano que sigue activo: el de detalles de la línea 140. La búsqueda interior ya había terminado antes de llegar aquí.

L178 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L179 |             reparto.agregar_remito(remito)
```

Con todos sus detalles cargados, agrega el remito al reparto. Por la sangría, esta línea está fuera del bucle de detalles y dentro del de remitos: se ejecuta una vez por remito terminado.

L180 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L181 |             if not confirmar(
```

Empieza la condición que pregunta si se desea otro remito. Se usa otra vez `not confirmar(...)` para salir cuando la respuesta sea NO.

```
L182 |                 "¿Querés agregar otro remito?"
```

Es el mensaje de la pregunta sobre otro remito. No pregunta por otro detalle: ese ciclo ya terminó.

```
L183 |             ):
```

Completa la condición. Si la respuesta fue negativa, su resultado es verdadero y se ejecutará el `break` siguiente.

```
L184 |                 break
```

Sale del bucle exterior de remitos iniciado en la línea 109. No sale de la función: todavía falta imprimir el reparto.

L185 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L186 a L190 · Mostrar todo y activar el programa

```
L186 |         mostrar_reparto(reparto)
```

Por su menor sangría está fuera del bucle de remitos. Llama una vez a la función que imprime el reparto completo.

L187 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

L188 · Línea en blanco. Separa visualmente ideas; no se ejecuta ni termina por sí misma un bloque.

```
L189 | if __name__ == "__main__":
```

`__name__` es una variable que Python establece para el módulo. Al ejecutar este archivo directamente vale `"__main__"` y la comparación da True. Si otro archivo lo importa normalmente, no se cumple esa igualdad y no se inicia la carga interactiva.

```
L190 |     CrearRepartoDiario.main()
```

Llama al método estático `main` de la clase de carga. Esta es la llamada que pone en marcha el programa interactivo. Los paréntesis vacíos indican que no necesita argumentos.

PASO 06

6. Cómo trabaja cada función

Para responder en el examen, usá tres preguntas: ¿qué recibe?, ¿qué hace?, ¿qué devuelve o modifica? La palabra efecto describe algo que cambia, como una lista o lo que se ve en pantalla.

Función o método	Entrada y trabajo	Resultado o efecto
`Articulo.__init__`	Recibe código, denominación y precio.	Inicializa los tres atributos. La expresión `Articulo(...)` produce la instancia; `__init__` no la devuelve explícitamente.
`RemitoVentaDetalle.__init__`	Recibe cantidad y artículo; rechaza cantidad menor o igual a cero.	Conserva los datos y guarda el subtotal.
`RemitoVenta.__init__`	Recibe cliente y número.	Crea un remito con lista propia y total cero.
`RemitoVenta.agregar_detalle`	Recibe un detalle y lo incorpora.	Modifica la lista y el total. Al terminar devuelve None.
`RemitoVenta.calcular_total`	Lee los subtotales de su lista.	Actualiza totalVenta y devuelve ese número.
`RepartoDiario.__init__`	Recibe fecha.	Crea un reparto con lista propia y total cero.
`RepartoDiario.agregar_remito`	Recibe un remito con detalles.	Modifica lista y total; puede lanzar ValueError. Si termina normalmente devuelve None.
`RepartoDiario.calcular_total`	Lee sus remitos y llama al cálculo de cada uno.	Actualiza totalReparto y devuelve ese número.
`leer_texto`	Recibe un mensaje; pide y recorta texto.	Devuelve una cadena no vacía.
`leer_entero_positivo`	Recibe un mensaje; convierte y valida.	Devuelve un entero mayor a cero.
`confirmar`	Recibe un mensaje; interpreta opciones de respuesta.	Devuelve True para SI y False para NO.
`buscar_articulo`	Recibe código y catálogo; compara uno por uno.	Devuelve el objeto coincidente o None.
`mostrar_reparto`	Recibe reparto; actualiza total y recorre datos.	Imprime el informe y devuelve None al terminar.

Función o método	Entrada y trabajo	Resultado o efecto
`CrearRepartoDiario.main`	Organiza el catálogo, clientes y carga por teclado.	Construye los objetos, los relaciona y muestra el resultado. Devuelve None al terminar.

Un viaje de ida y vuelta

```
numero = leer_entero_positivo("Número de remito: ")
```

1. main envía el texto de la pregunta a la función.
2. Dentro de la función, ese texto se usa mediante el parámetro mensaje.
3. input recibe lo escrito, por ejemplo la cadena "2".
4. int la convierte en el entero 2.
5. La comparación comprueba que 2 sea mayor a cero.
6. return entrega 2 y termina la función.
7. main recibe el resultado y lo guarda en su variable numero.

Las variables locales pertenecen a la llamada donde se usan. Que aparezca el nombre `numero` en main y dentro de la función no significa que sea una única variable compartida. El valor vuelve gracias a `return`.

Tres formas distintas de salir

Instrucción	Qué termina	Ejemplo de este trabajo
`return`	Toda la llamada actual a la función.	leer_texto deja de pedir y entrega el texto.
`break`	El bucle for o while más cercano.	Código válido: termina la búsqueda, pero sigue el detalle.
`continue`	Solo la vuelta actual del bucle más cercano.	Entrada que no se convierte: vuelve a pedir el entero.

Lo que pasa con una excepción

En `int(input(...))`, escribir "hola" puede lanzar ValueError. El `except ValueError` de esa función captura ese problema, muestra el aviso y reintenta. En cambio, `raise ValueError(...)` crea deliberadamente un error por una regla del modelo, como cantidad cero.

La carga normal pide una cantidad válida antes de crear el detalle y agrega el remito solo después de cargar detalles. Si se llaman las clases directamente con datos inválidos y nadie captura la excepción, la ejecución muestra el error y se interrumpe. Esa diferencia explica por qué están las validaciones de entrada y también las del modelo.

PASO 07

7. Un reparto completo, sin saltar pasos

Vamos a cargar dos remitos. El primero corresponde a un cliente del diccionario. El segundo usa un CUIT nuevo, por lo que se pide su nombre. Los valores monetarios y la salida de este ejemplo se comprobaron ejecutando tu programa.

Paso	Qué se pide	Qué escribís
1	Fecha	03/11/2026
2	Número del primer remito	1
3	CUIT del cliente conocido	20284569875
4	Código de artículo	10
5	Cantidad	2
6	¿Otro detalle?	SI
7	Código de artículo	20
8	Cantidad	1
9	¿Otro detalle?	NO
10	¿Otro remito?	SI
11	Número del segundo remito	2
12	CUIT nuevo	20999999999
13	Razón social del cliente nuevo	Taller Ejemplo
14	Código de artículo	30
15	Cantidad	3
16	¿Otro detalle?	NO
17	¿Otro remito?	NO

Cómo cambian los importes en memoria

Momento	Cambio que ocurre
Se crea el reparto	remitosVenta está vacía y totalReparto vale 0.0.
Se crea el remito 1	Tiene cliente Juan Alonso, número 1, lista de detalles vacía y totalVenta 0.0.
Detalle: 2 filtros de aceite	2 × 10000 = 20000. Se agrega el detalle y totalVenta del remito 1 pasa a 20000.
Detalle: 1 filtro de aire	1 × 8000 = 8000. El recálculo empieza en cero: 0 + 20000 + 8000 = 28000.
Se agrega el remito 1	El reparto tiene un remito y totalReparto pasa a 28000.
Se registra el cliente nuevo	El diccionario pasa de cinco a seis claves y asocia el CUIT nuevo con Taller Ejemplo.
Se carga el remito 2	3 × 7500 = 22500. Su totalVenta es 22500. El reparto aún no lo sumó mientras se está cargando.
Se agrega el remito 2	El reparto recalcula desde cero: 28000 + 22500 = 50500.

Por qué el recálculo no duplica la venta

```
self.totalVenta = 0.0
for detalle in self.detallesRemito:
    self.totalVenta += detalle.subtotal
```

Cada llamado reconstruye la suma completa desde cero. Si la lista contiene subtotales de 20000 y 8000, llamar dos veces devuelve 28000 las dos veces. Sin el reinicio se acumularía sobre el cálculo previo: ese sería un error.

El informe final real de esa carga

```
Reparto Diario
Fecha: 03/11/2026

-------------- Remitos del Reparto --------------

Nombre Cliente: Juan Alonso
Número Remito: 1
Cantidad Denominación Artículo          Precio Unitario     Subtotal
       2 Filtro de Aceite                      10000.00     20000.00
       1 Filtro de Aire                         8000.00      8000.00
Total Remito Venta: $28000.00
---------------------------------------------------------------------

Nombre Cliente: Taller Ejemplo
Número Remito: 2
Cantidad Denominación Artículo          Precio Unitario     Subtotal
       3 Filtro de Combustible                  7500.00     22500.00
Total Remito Venta: $22500.00
---------------------------------------------------------------------

Monto Total Reparto: $50500.00
```

Qué objetos se comparten

El catálogo guarda un objeto para cada producto. `buscar_articulo` devuelve una referencia a uno de esos objetos. El detalle conserva esa referencia. Si dos detalles usan el mismo artículo, cada uno conserva su cantidad y subtotal, pero ambos pueden referenciar el mismo objeto del catálogo.

El subtotal se calcula y se guarda al construir el detalle. Esta versión no observa automáticamente cambios posteriores del precio: recalcular un remito suma los subtotales guardados. El ejercicio carga ventas con el catálogo establecido y no incluye una operación para editar esos precios.

PASO 08

8. Bucles y validaciones con ejemplos

Los tres niveles de carga

Bucle de main	Líneas	Cómo termina
Exterior: remitos	109 a 184	El break de L184 se ejecuta al responder NO a otro remito.
Intermedio: detalles	140 a 177	El break de L177 se ejecuta al responder NO a otro detalle.
Interior: código válido	143 a 157	El break de L153 se ejecuta cuando la búsqueda devuelve un artículo.

Dentro de esos recorridos también se llaman funciones auxiliares con sus propios bucles. Al llamar a leer_entero_positivo, por ejemplo, sus reintentos ocurren dentro de esa función. Un `return` de la auxiliar vuelve a main sin terminar main.

Código 99: cómo vuelve a pedirlo

1. Se guarda "99" en codigo.
2. La función buscar_articulo revisa el catálogo completo.
3. No encuentra coincidencias y devuelve None.
4. `articulo is not None` da False.
5. No se ejecuta break; se imprime el aviso.
6. El bucle interior vuelve a pedir otro código.
7. Si se ingresa "10", se devuelve un objeto y el break permite continuar con la cantidad.

Cantidad inválida: no todos los errores son iguales

Entrada	Qué ocurre
`hola`	int no puede convertirla. ValueError lleva al except y continue vuelve a pedir.
`0`	Se convierte en entero, pero no es mayor a cero. Se muestra el aviso y se repite.
`-2`	También se convierte, pero falla la condición de positividad.
`2.5`	Es una cadena decimal y esta llamada a int no la acepta. Se vuelve a pedir un entero.
2	Se convierte, cumple la condición y return entrega el entero 2.

La respuesta negativa y el operador not

Respuesta reconocida	confirmar devuelve	not invierte	¿Se ejecuta break?
SI	True	False	No: puede cargarse otro elemento.
NO	False	True	Sí: termina ese bucle.

Cómo se garantiza el primer detalle

El bucle de detalles entra directamente porque usa `while True`. Dentro primero obliga a encontrar artículo, pedir cantidad y agregar el detalle. Recién después pregunta si se quiere otro. Además, RepartoDiario.agregar_remito comprueba la lista para proteger la regla al usar los objetos desde otro código.

Qué comprueba realmente esta versión

Texto no vacío; número y cantidad como enteros positivos; artículo existente; opciones de confirmación reconocidas; y remito con al menos un detalle. La fecha y el CUIT se manejan como texto no vacío. No se verifica un formato de calendario ni el dígito verificador del CUIT. Tampoco se verifica que los números de remito sean únicos: no lo solicita esta consigna.

Los objetos y el diccionario viven en memoria durante la ejecución. El cliente nuevo se reutiliza si se ingresa de nuevo en esa ejecución. Al cerrar el programa no queda guardado en una base de datos.

PASO 09

9. Errores que conviene poder reconocer

Error frecuente	Por qué falla	Qué hace esta solución
Confundir = con ==	= asigna; == compara.	Asigna atributos con = y compara códigos con ==.
Buscar "10" como si fuera 10	Cadena y entero no son el mismo valor.	El catálogo y la entrada usan códigos como cadenas.
Devolver None dentro del for	Podría abandonar la búsqueda después del primer artículo que no coincide.	El return None está después del for.
Crear una lista compartida en la clase	Los objetos podrían mezclar datos si comparten la misma lista.	Crea listas propias dentro de __init__.
Olvidar reiniciar el acumulador	El nuevo cálculo se suma sobre el total anterior.	Comienza calcular_total con 0.0.
Guardar el resultado de append	append devuelve None, aunque la lista se haya modificado.	Llama a append sin reemplazar la lista con su resultado.
Preguntar antes del primer detalle	Podría permitir un remito vacío.	Pregunta por otro detalle después de incorporar el primero.
Poner el total general dentro del for	Se imprimiría por cada remito.	mostrar_reparto lo deja fuera de los dos bucles.
Creer que print entrega el total	print muestra y normalmente devuelve None.	Los cálculos usan return para entregar números.
Pensar que main es automático	Su nombre no lo ejecuta por sí solo.	La última línea llama explícitamente al método.

Cómo explicarlo oralmente

«Representé los productos con Articulo. Cada detalle guarda una cantidad y referencia un artículo; su subtotal es cantidad por precio. Un remito tiene una lista propia de detalles y suma sus subtotales. El reparto tiene su lista de remitos y suma sus totales. El main resuelve el cliente con un diccionario, busca artículos en una lista y organiza la carga con bucles. Las preguntas de continuación aparecen después de cargar, para asegurar el mínimo pedido».

PASO 10

10. Practicá como en el examen

Intentá responder primero sin mirar las soluciones. Para los cálculos, dibujá las listas y anotá el acumulador después de cada vuelta.

1. Una ficha de producto

¿Qué atributos quedan al ejecutar Articulo("20", "Filtro de Aire", 8000)?

2. Un detalle

Se venden 3 unidades del artículo "20". ¿Qué valor queda en subtotal y dónde se obtiene el precio?

3. Un remito

Tiene detalles de $20000 y $24000. ¿Cuánto devuelve calcular_total? ¿Y si se llama otra vez sin cambios?

4. Un reparto

Tiene remitos por $44000 y $25000. ¿Cuál es totalReparto? ¿Qué método se reutiliza?

5. Dos tipos

¿Por qué "10" == 10 da False y cómo evita el programa esa confusión?

6. Las salidas

Explicá qué terminan return, break y continue en este programa.

7. Un CUIT nuevo

¿Qué línea agrega el cliente? ¿Qué ocurre si se ingresa ese CUIT en el remito siguiente?

8. Una búsqueda

Si se busca el código "50", ¿por qué return None debe estar fuera del for?

9. Una cantidad cero

¿Llega un cero escrito por teclado a crear un detalle? ¿Y qué pasa si se llama directamente a RemitoVentaDetalle(0, articulo)?

10. Una lista propia

¿En qué lugar se crea detallesRemito y por qué importa ese lugar?

11. Un formato

Explicá cada parte de :>15.2f. ¿Cambia el precio guardado?

12. Una ejecución

¿Qué controla if __name__ == "__main__"? ¿Qué diferencia hay entre ejecutar el archivo e importarlo?

13. Un resultado ausente

¿Qué devuelve buscar_articulo si no encuentra un código? ¿Qué devuelve mostrar_reparto al terminar normalmente?

14. Una condición invertida

Si confirmar devuelve False, ¿qué valor toma not confirmar(...) y por qué se hace break?

Ejercicio de seguimiento de entrada

Escribís estas cantidades en orden: hola, 0, -1 y 2. Explicá el camino de cada una por try, except, continue, if y return. Después repetí el recorrido para las respuestas: quizá y NO.

Reconstrucción desde una hoja en blanco

1. Dibujá la relación de los cuatro objetos del negocio.
2. Escribí Articulo y probá una instancia.
3. Escribí el detalle y comprobá un subtotal.
4. Escribí RemitoVenta y su suma.
5. Escribí RepartoDiario y reutilizá la suma de los remitos.
6. Prepará catálogo y diccionario.
7. Resolvé primero un cliente y un detalle; después agregá los bucles.
8. Probá código inválido, cantidad inválida y cliente nuevo.
9. Ubicá cada pregunta de continuación después de la carga correspondiente.
10. Recién entonces armá la presentación en columnas.

Respuestas para comprobarte

1. codigo es "20", denominacion es "Filtro de Aire" y precio es 8000.

2. subtotal es 3 × 8000 = 24000. El precio se obtiene del objeto articulo recibido por el detalle.

3. Devuelve 44000 las dos veces, porque empieza otra vez desde 0.0.

4. totalReparto es 69000. Se reutiliza remito.calcular_total() dentro del cálculo del reparto.

5. Uno es texto y el otro es entero. Los códigos del catálogo y los obtenidos por input se mantienen como texto.

6. return termina la función llamada; break termina el bucle más cercano; continue empieza otra vuelta de ese bucle.

7. clientesReparto[cuit] = nombreCliente agrega la asociación. En el siguiente remito de esa ejecución, la clave ya existe y se recupera el nombre.

8. Los primeros artículos pueden no coincidir y todavía falta revisar el último. Si se devolviera None al fallar la primera comparación, no se alcanzaría el código "50".

9. Por teclado, cero se rechaza y se vuelve a pedir. En la llamada directa, el inicializador lanza ValueError antes de asignar los atributos de la venta.

10. Dentro de __init__, con self.detallesRemito = []. Así cada remito tiene su propia lista.

11. > alinea a la derecha; 15 es ancho mínimo; .2 pide dos decimales; f indica formato fijo. Solo afecta la representación.

12. Ejecutar directamente hace que __name__ valga "__main__" y se llame a main. Importar normalmente carga definiciones sin iniciar la interacción de ese bloque.

13. La búsqueda devuelve None explícitamente. mostrar_reparto también devuelve None al terminar, porque no tiene return explícito.

14. not False vale True. Por eso se entra al if y se ejecuta break para terminar la carga correspondiente.

Seguimiento resuelto

hola: int falla, except muestra el aviso y continue vuelve a pedir.
0: int funciona, pero numero > 0 da False; se muestra el aviso y vuelve el while.
-1: ocurre lo mismo que con cero.
2: int funciona, la condición da True y return entrega el entero.
quizá: confirmar la convierte a mayúsculas, no reconoce la opción y repite.
NO: coincide con una opción negativa y return entrega False.

PASO 11

11. Los cinco archivos completos

Este apéndice reúne el código exacto que se explicó. Los números de línea se agregan solo en el PDF y no deben copiarse como parte de Python. El informe se basó en los archivos existentes, sin reescribirlos.

Articulo.py

Articulo.py · líneas 1 a 5

```
001  class Articulo:
002      def __init__(self, codigo, denominacion, precio):
003          self.codigo = codigo
004          self.denominacion = denominacion
005          self.precio = precio
```

RemitoVentaDetalle.py

RemitoVentaDetalle.py · líneas 1 a 8

```
001  class RemitoVentaDetalle:
002      def __init__(self, cantidad, articulo):
003          if cantidad <= 0:
004              raise ValueError("La cantidad debe ser mayor a cero.")
005
006          self.cantidad = cantidad
007          self.articulo = articulo
008          self.subtotal = cantidad * articulo.precio
```

RemitoVenta.py

RemitoVenta.py · líneas 1 a 18

```
001  class RemitoVenta:
002      def __init__(self, nombreCliente, numeroRemito):
003          self.nombreCliente = nombreCliente
004          self.numeroRemito = numeroRemito
005          self.detallesRemito = []
006          self.totalVenta = 0.0
007
008      def agregar_detalle(self, detalle):
009          self.detallesRemito.append(detalle)
010          self.calcular_total()
011
012      def calcular_total(self):
013          self.totalVenta = 0.0
014
015          for detalle in self.detallesRemito:
016              self.totalVenta += detalle.subtotal
017
018          return self.totalVenta
```

RepartoDiario.py

RepartoDiario.py · líneas 1 a 22

```
001  class RepartoDiario:
002      def __init__(self, fecha):
003          self.fecha = fecha
004          self.remitosVenta = []
005          self.totalReparto = 0.0
006
007      def agregar_remito(self, remito):
008          if not remito.detallesRemito:
009              raise ValueError(
010                  "El remito debe tener al menos un detalle."
011              )
012
013          self.remitosVenta.append(remito)
014          self.calcular_total()
015
016      def calcular_total(self):
017          self.totalReparto = 0.0
018
019          for remito in self.remitosVenta:
020              self.totalReparto += remito.calcular_total()
021
022          return self.totalReparto
```

CrearRepartoDiario.py

CrearRepartoDiario.py · líneas 1 a 37

```
001  from Articulo import Articulo
002  from RemitoVentaDetalle import RemitoVentaDetalle
003  from RemitoVenta import RemitoVenta
004  from RepartoDiario import RepartoDiario
005
006
007  def leer_texto(mensaje):
008      while True:
009          texto = input(mensaje).strip()
010
011          if texto:
012              return texto
013
014          print("El texto no puede quedar vacío.")
015
016
017  def leer_entero_positivo(mensaje):
018      while True:
019          try:
020              numero = int(input(mensaje))
021          except ValueError:
022              print("Ingresá un número entero.")
023              continue
024
025          if numero > 0:
026              return numero
027
028          print("El número debe ser mayor a cero.")
029
030
031  def confirmar(mensaje):
032      while True:
033          respuesta = input(
034              f"{mensaje} (SI/NO): "
035          ).strip().upper()
036
037          if respuesta in ("SI", "SÍ", "S"):
```

CrearRepartoDiario.py · líneas 38 a 74

```
038             return True
039
040         if respuesta in ("NO", "N"):
041             return False
042
043         print("Respondé SI o NO.")
044
045
046 def buscar_articulo(codigo, articulosDisponibles):
047     for articulo in articulosDisponibles:
048         if articulo.codigo == codigo:
049             return articulo
050
051     return None
052
053
054 def mostrar_reparto(reparto):
055     reparto.calcular_total()
056
057     print("\nReparto Diario")
058     print(f"Fecha: {reparto.fecha}")
059     print("\n-------------- Remitos del Reparto --------------")
060
061     for remito in reparto.remitosVenta:
062         print(f"\nNombre Cliente: {remito.nombreCliente}")
063         print(f"Número Remito: {remito.numeroRemito}")
064
065         print(
066             f"{'Cantidad':>8} "
067             f"{'Denominación Artículo':<30} "
068             f"{'Precio Unitario':>15} "
069             f"{'Subtotal':>12}"
070         )
071
072         for detalle in remito.detallesRemito:
073             print(
074                 f"{detalle.cantidad:>8} "
```

CrearRepartoDiario.py · líneas 75 a 111

```
075                 f"{detalle.articulo.denominacion:<30} "
076                 f"{detalle.articulo.precio:>15.2f} "
077                 f"{detalle.subtotal:>12.2f}"
078             )
079 
080         print(f"Total Remito Venta: ${remito.totalVenta:.2f}")
081         print("-" * 70)
082 
083     print(f"\nMonto Total Reparto: ${reparto.totalReparto:.2f}")
084 
085 
086 class CrearRepartoDiario:
087     @staticmethod
088     def main():
089         articulosDisponibles = [
090             Articulo("10", "Filtro de Aceite", 10000),
091             Articulo("20", "Filtro de Aire", 8000),
092             Articulo("30", "Filtro de Combustible", 7500),
093             Articulo("40", "Aceite de Motor sw/10", 25000),
094             Articulo("50", "Correa de Distribución", 20000),
095         ]
096 
097         clientesReparto = {
098             "20284569875": "Juan Alonso",
099             "20157896542": "Emiliano Salas",
100             "50258963654": "Pacallcar Autos",
101             "27654987456": "Julieta Videla",
102             "50335588945": "Todo Auto SRL",
103         }
104 
105         fecha = leer_texto("Fecha del reparto: ")
106         reparto = RepartoDiario(fecha)
107 
108         # Bucle de carga de remitos.
109         while True:
110             print("\nNUEVO REMITO")
111 
```

CrearRepartoDiario.py · líneas 112 a 148

```
112             numero = leer_entero_positivo(
113                 "Número de remito: "
114             )
115
116             cuit = leer_texto("CUIT del cliente: ")
117
118             if cuit in clientesReparto:
119                 nombreCliente = clientesReparto[cuit]
120                 print(f"Cliente: {nombreCliente}")
121             else:
122                 nombreCliente = leer_texto(
123                     "Cliente nuevo. Ingresá su razón social: "
124                 )
125                 clientesReparto[cuit] = nombreCliente
126
127             remito = RemitoVenta(nombreCliente, numero)
128
129             print("\nARTÍCULOS DISPONIBLES")
130
131             for articulo in articulosDisponibles:
132                 print(
133                     f"{articulo.codigo} - "
134                     f"{articulo.denominacion} - "
135                     f"${articulo.precio:.2f}"
136                 )
137
138             # Bucle de carga de detalles.
139             # La pregunta de salida aparece después del primer detalle.
140             while True:
141
142                 # Repetir la búsqueda hasta ingresar un código válido.
143                 while True:
144                     codigo = leer_texto(
145                         "Código del artículo: "
146                     )
147
148                     articulo = buscar_articulo(
```

CrearRepartoDiario.py · líneas 149 a 185

```
                        codigo, articulosDisponibles
                    )

                    if articulo is not None:
                        break

                    print(
                        "el código es incorrecto, ingrese nuevamente"
                    )

                cantidad = leer_entero_positivo(
                    "Cantidad: "
                )

                detalle = RemitoVentaDetalle(
                    cantidad, articulo
                )

                remito.agregar_detalle(detalle)

                print(
                    f"Subtotal del detalle: "
                    f"${detalle.subtotal:.2f}"
                )

                if not confirmar(
                    "¿Querés agregar otro detalle?"
                ):
                    break

            reparto.agregar_remito(remito)

            if not confirmar(
                "¿Querés agregar otro remito?"
            ):
                break

```

CrearRepartoDiario.py · líneas 186 a 190

```
        mostrar_reparto(reparto)


if __name__ == "__main__":
    CrearRepartoDiario.main()
```

PASO 12

12. Cómo ejecutar y qué material se usó

Abrí una terminal en la raíz del repositorio C:\UTN-TUPaD-Programacion1. El punto de entrada de esta práctica es CrearRepartoDiario.py. Los otros cuatro archivos deben estar en su misma carpeta, porque se importan desde allí.

```
python "13 - Practica Integradora/CrearRepartoDiario.py"
```

También podés usar el intérprete del entorno virtual ya creado en el repositorio. En PowerShell:

```
& ".\.venv\Scripts\python.exe" `
  ".\13 - Practica Integradora\CrearRepartoDiario.py"
```

La comilla invertida al final de la primera línea es una continuación de PowerShell. Es parte de ese comando de terminal, no una instrucción del programa Python. Esta práctica utiliza solamente la biblioteca estándar de Python y no requiere instalar Flask.

Material de referencia

Consigna: PRACTICA INTEGRADORA.pdf, proporcionada por vos. Define las clases, el catálogo, el diccionario, las validaciones y la salida esperada.

Código explicado: los cinco archivos de la carpeta 13 - Practica Integradora, leídos el 03/11/2026. Si después editás el código, sus números de línea pueden cambiar. Esta edición explica el contenido observado en esa fecha.

Relación con lo estudiado: el subtotal usa la misma idea de cantidad por precio del TP de computadoras; las listas anidadas y el mínimo de una carga siguen la lógica del TP de alumnos y notas; la suma delegada se parece al barrio que reutiliza el cálculo de cada vivienda.

Comprobación del material

Se anotaron las 243 líneas de los cinco archivos, incluidas las líneas vacías, y se cubrieron los 68 nombres distintos encontrados por el analizador de tokens. Se verificaron sintaxis, cálculos del ejemplo, reintentos, búsqueda, listas independientes, remito vacío y reutilización de un cliente nuevo.

El total de $50500 del ejemplo se obtuvo ejecutando el programa con las entradas indicadas. Las comprobaciones se hicieron sin modificar los cinco archivos fuente.

Antes de rendir

Elegí una línea al azar y explicá: qué nombres usa, qué tipo de dato espera, qué modifica y hacia dónde sigue la ejecución. Cuando podés hacer eso y seguir el ejemplo sin mirar las respuestas, estás comprendiendo la solución.